Transform Your Future with
"HELLO WORLD!"
INSTITUTE

Why learn with us ?

Training From Experts

One - on – One Guidance

100% Practical

Real-Time Projects & Case Studies

Placement Preparation Support

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Contact :- 8085896740 @ hello_world_institute Hello World Institute

Address : 2nd Floor , Near KFC , Tower Square , Bhawarkua , Indore

Sets

- Sets are used to store multiple items in a single variable.
- Mathematically a set is a collection of items not in any particular order.
- There is no index attached to any element in a python set. So they do not support any indexing or slicing operation.

```
set1 = {"apple", "banana", "cherry"}
print(set1)
```

Note: Sets are unordered, so you cannot be sure in which order the items will appear.

Set Items

Set items are unordered, unchangeable, and do not allow duplicate values.

Unordered

- Unordered means that the items in a set do not have a defined order.
- Set items can appear in a different order every time you use them, and cannot be referred to by index or key.

Unchangeable

- Set items are unchangeable, meaning that we cannot change the items after the set has been created.
- Once a set is created, you cannot change its items, but you can remove items and add new items.

Duplicates Not Allowed

Sets cannot have two items with the same value.

Example :

Duplicate values will be ignored:

```
set1 = {"apple", "banana", "cherry", "apple"}

print(set1)
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Get the Length of a Set

To determine how many items a set has, use the len() function.

Example : Get the number of items in a set:

```
set1 = {"apple", "banana", "cherry"}
print(len(set1))
```

Set Items - Data Types

Set items can be of any data type:

Example :String, int and boolean data types:

```
set1 = {"apple", "banana", "cherry"}
set2 = {1, 5, 7, 9, 3}

set3 = {True, False, False}
```

A set can contain different data types:

Example :A set with strings, integers and boolean values:

```
set1 = {"abc", 34, True, 40, "male"}
```

type()

From Python's perspective, sets are defined as objects with the data type 'set':

<class 'set'>

Example

```
myset = {"apple", "banana", "cherry"}

print(type(myset))
```

The set() Constructor

It is also possible to use the set() constructor to make a set.

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Example: Using the set() constructor to make a set:

```
set1 = set(("apple", "banana", "cherry")) # note the double round-brackets
print(set1)
```

Access Items

- You cannot access items in a set by referring to an index or a key.
- But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the in keyword.

Example : Loop through the set, and print the values:

```
set1 = {"apple", "banana", "cherry"}
for x in set1:   print(x)
```

Example : Check if "banana" is present in the set:

```
set1 = {"apple", "banana", "cherry"}
print("banana" in set1)
```

Change Items

Once a set is created, you cannot change its items, but you can add new items.

Add Items

- Once a set is created, you cannot change its items, but you can add new items.
- To add one item to a set use the add() method.

Example : Add an item to a set, using the add() method:

```
set1 = {"apple", "banana", "cherry"}
set1.add("orange") print(set1)
```

Add Sets

To add items from another set into the current set, use the update() method.

Example : Add elements from tropical into thisset:

```
set1 = {"apple", "banana", "cherry"}

tropical = {"pineapple", "mango", "papaya"}
set1.update(tropical)

print(set1)
```

Add Any Iterable

The object in the update() method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

Example : Add elements of a list to at set:

```
set1 = {"apple", "banana", "cherry"}
mylist = ["kiwi", "orange"]
set1.update(mylist)

print(set1)
```

Remove Item

- To remove an item in a set, use the remove (), or the discard () method.

Example : Remove "banana" by using the remove() method:

```
set1 = {"apple", "banana", "cherry"}
set1.remove("banana")

print(set1)
```

Note: If the item to remove does not exist, remove () will raise an error.

Example: Remove "banana" by using the discard () method:

```
set1 = {"apple", "banana", "cherry"}
set1.discard("banana")

print(set1)
```

Note: If the item to remove does not exist, discard () will **NOT** raise an error.

You can also use the pop () method to remove an item, but this method will remove the *last* item. Remember that sets are unordered, so you will not know what item that gets removed.
The return value of the pop () method is the removed item.

Example: Remove the last item by using the pop() method:

```
set1 = {"apple", "banana", "cherry"}
x = set1.pop()

 print(x)

print(set1)
```

Note: Sets are *unordered*, so when using the pop () method, you do not know which item that gets removed.

Example: The clear () method empties the set:

```
set1 = {"apple", "banana", "cherry"}
set1.clear()

print(set1)
```

Example: The del keyword will delete the set completely:

```
set1 = {"apple", "banana", "cherry"}
del set1

print(set1)
```

Loop Items

You can loop through the set items by using a for loop:

Example: Loop through the set, and print the values:

```
set1 = {"apple", "banana", "cherry"}
for x in set1:

    print(x)
```

Join Two Sets

There are several ways to join two or more sets in Python.
You can use the union () method that returns a new set containing all items from both sets, and the update () method that inserts all the items from one set into another:

Example: The union() method returns a new set with all items from both sets:

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}

set3=set1.union(set2)
print(set3)
```

Example: The update() method inserts the items in set2 into set1:

```
set1 = {"a", "b" , "c"}

 set2 = {1, 2, 3}
set1.update(set2)
print(set1)
```

Note: Both union () and update() will exclude any duplicate items.

Keep ONLY the Duplicates

The intersection_update() method will keep only the items that are present in both sets.

Example: Keep the items that exist in both set x, and set y:

```
x = {"apple", "banana", "cherry"}

y = {"google", "microsoft", "apple"}
x.intersection_update(y)

 print(x)
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

The intersection () method will return a *new* set, that only contains the items that are present in both sets.

Example

Return a set that contains the items that exist in both set x, and set y:

```
x = {"apple", "banana", "cherry"}

y = {"google", "microsoft", "apple"}
z = x.intersection(y)

print(z)
```

Keep All, But NOT the Duplicates

The symmetric_difference_update() method will keep only the elements that are NOT present in both sets.

Example: Keep the items that are not present in both sets:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
x.symmetric_difference_update(y)
print(x)
```

The symmetric_difference() method will return a new set, that contains only the elements that are NOT present in both sets.

Example: Return a set that contains all items from both sets, except items that are present in both:

```
x = {"apple", "banana", "cherry"}

y = {"google", "microsoft", "apple"}
z = x.symmetric_difference(y)
print(z)
```

Set Methods

Python has a set of built-in methods that you can use on sets.

Method	Description
add()	Adds an element to the set
clear()	Removes all the elements from the set
copy()	Returns a copy of the set
difference()	Returns a set containing the difference between two or more sets
difference_update()	Removes the items in this set that are also included in another, specified set
discard()	Remove the specified item
intersection()	Returns a set, that is the intersection of two other sets
intersection_update()	Removes the items in this set that are not present in other, specified set(s)
isdisjoint()	Returns whether two sets have a intersection or not

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
pop()	Removes an element from the set
remove()	Removes the specified element
symmetric_difference()	Returns a set with the symmetric differences of two sets
symmetric_difference_update()	inserts the symmetric differences from this set and another
union()	Return a set containing the union of sets
update()	Update the set with the union of this set and others

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS